

09. One-to-Few & One-to-Many Relationships

📌 Relational SQL vs NoSQL Data Modeling

While SQL relies strictly on Foreign Keys and JOINS, MongoDB offers two foundational paradigms: **Embedding (Denormalization)** and **Referencing (Normalization)**.

Approach 1: One-to-Few (Embedded Documents)

Ideal when child documents are small in number and rarely grow (e.g. User with 2-3 addresses). Child documents are stored directly in an array inside the parent:

```
One-to-Few Schema - Embedding

const userSchema = new mongoose.Schema({
  username: String,
  addresses: [
    { _id: false, location: String, city: String } // Embedded directly inside user!
  ]
});
```

Approach 2: One-to-Many (Child Referencing)

Used when child items can grow substantially (e.g. Customer with multiple Orders). The parent document stores an array of child **ObjectId** references:

```
One-to-Many Schema - Referencing

// 1. Order Schema
const orderSchema = new mongoose.Schema({ item: String, price: Number });
const Order = mongoose.model("Order", orderSchema);

// 2. Customer Schema (Stores Array of ObjectIds)
const customerSchema = new mongoose.Schema({
  name: String,
  orders: [{ type: mongoose.Schema.Types.ObjectId, ref: "Order" }]
});
const Customer = mongoose.model("Customer", customerSchema);
```

🚀 The Power of populate()

.populate("orders") replaces those raw ObjectIds with actual Order documents!

```
Query with populate()

const customer = await Customer.findOne({ name: "Rahul" }).populate("orders");
console.log(customer);
```

One-to-Squillions & Architecture Summary

Approach 3: One-to-Squillions (Parent Referencing)

Why is this needed? In MongoDB, every document has a hard size limit of **16 MB**. If a Post has 1,000,000 comments or an influencer has 10 million followers, storing ObjectIds in an array inside the parent document will exceed 16 MB and crash!

 **16 MB DOCUMENT LIMIT:** Never store unbounded arrays (thousands/millions of items) inside a single document! Always utilize Parent Referencing!

Solution: Store the Parent reference's ObjectId inside each Child document:

```

// 1. Post Schema (Contains NO comments array!)
const postSchema = new mongoose.Schema({ title: String, content: String });

// 2. Comment Schema (Stores Reference to Parent Post)
const commentSchema = new mongoose.Schema({
  text: String,
  post: { type: mongoose.Schema.Types.ObjectId, ref: "Post" } // 🐘 Parent reference
});
  
```

Master Relationship Decision Matrix

Relationship Type	Volume Scale	Schema Design Pattern	Real-world Use Case
One-to-Few	Few items (< 10)	Embedded (Denormalized inside parent)	User addresses, phone numbers
One-to-Many	Medium (10 to ~1000)	Child Referencing (Array of IDs in parent)	Customer orders, shopping cart items
One-to-Squillions	Massive (1,000 to Millions)	Parent Referencing (Parent ID inside child)	Post comments, system logs, followers

Topper's Quick Revision Checklist

- ✓ **mongosh:** use `db.col.insertOne()`, `db.col.find()`, `db.col.updateOne()`.
- ✓ **BSON:** Binary JSON featuring rich data types (Date, ObjectId) & fast indexed traversal.
- ✓ **Mongoose Model:** "User" model creates "users" collection automatically.
- ✓ **Updates:** Always pass `{ runValidators: true }` on `findByIdAndUpdate!`
- ✓ **wrapAsync:** `fn(req,res,next).catch(next)` replaces messy try-catch blocks.
- ✓ **Populate:** Swaps referenced ObjectIds with actual linked documents during query.